

Thinking Outside the Box:

Retention and Transmission of Information in Sliding-Window KV Inference

Timothy DeLise* Seth Cromelin

August 2026

Abstract

Sliding-window KV inference refers to processing a sequence incrementally while retaining only a fixed-size cache of recent key and value states. It can be applied to pretrained causal transformers at inference time without additional training, while its KV-cache memory remains fixed as more tokens are processed. Because cached states are computed in the context of earlier tokens, they may carry information from beyond the current window and transmit it to later states. This study presents a series of experiments using five open-weight models spanning Qwen, Llama, Mistral, and Muse Glimmer. We investigate whether information originating outside the immediate context window can persist through a rolling KV cache and remain useful for retrieval. Initial results show that retaining previously computed states improves retrieval across the models tested compared with recomputing the final fixed window from raw tokens. We then measure how far this effect extends and find that Muse Glimmer and Mistral 7B show the strongest *latent information relay*: they can recover information even after the relevant source tokens have left the cache. Both models incorporate sliding-window attention in their published architectures, an association that motivates testing whether training with sliding windows promotes more reliable information retention.

1 Introduction

This study investigates *sliding-window KV inference*, autoregressive inference using a fixed-width key-value (KV) cache. At each generation step, the oldest cached position is discarded, a new position is appended, and activations are computed only for the newly appended token. Variants of this mechanism are used in models such as Mistral 7B (Jiang et al. 2023 [8]), which uses sliding-window attention throughout the model, and Muse Glimmer (Meta AI 2026 [10]), which interleaves sliding-window and full-attention layers. We evaluate information persistence under Rolling-KV inference across five models: Qwen2.5 0.5B and 3B (Team 2025 [13]), Mistral 7B (Jiang et al. 2023 [8]), Llama 3.1 8B (Dubey et al. 2024 [4]), and Muse Glimmer 30B (Meta AI 2026 [10]), spanning multiple model architectures and scales. Importantly, the rolling KV cache is imposed at inference time on an otherwise standard pretrained causal transformer, without requiring additional pretraining or architectural modification. The appeal of sliding-window KV inference is that it combines a fixed memory footprint with the possibility that representations inside the current window may carry information originating from tokens that have already left the cache. In this sense, the transformer

*Corresponding author: timothy.delise@umontreal.ca

becomes a bounded-state recurrent computation: past information need not remain explicitly accessible if it can be propagated forward through successive token representations. This raises the possibility of a simple streaming approach to long-context inference. We test this hypothesis directly, asking whether information originating outside the current KV window can propagate into it, how much information survives, and over what distances. We study these questions through three experiments comparing sliding-window KV inference with standard fixed-window inference.

The mechanism underlying this possibility follows directly from causal transformer computation (Vaswani et al. 2017 [14]). At each layer, attention is the only operation that mixes information across token positions: the residual-stream activation at position i may incorporate information from positions $1, \dots, i$, after which the MLP acts independently at that position. Repeating this computation through depth allows information originating at an earlier token to become materialized in the residual-stream activations of later tokens. Section 3.1 formalizes this computation, and Figure 1 illustrates the corresponding information flow.

This creates a distinction between the position at which information originates and the cached states in which its influence is subsequently represented. Suppose information originating at token t_i is incorporated into the residual-stream activation of a later token t_j .

Under ordinary fixed-window inference, once the window shifts and its contents are recomputed without t_i , any dependence on that token is lost. Under sliding-window KV inference, however, the already-computed cached state associated with t_j is retained. Thus, when t_i is eventually evicted, information originating there may remain materialized in cached states that are still inside the window.

The central empirical question is therefore: how much of this information survives, and for how long?

This question is relevant to the broader problem of long-context inference. Standard transformer inference requires memory that grows with the amount of retained context, whereas recurrent architectures maintain a state whose size is independent of sequence length. State-space models such as Mamba (Gu and Dao 2023 [7]) have renewed interest in this recurrent paradigm, while recent hybrid architectures retain attention alongside recurrent state-space layers to balance bounded-memory sequence processing with the capabilities of attention (NVIDIA 2025 [11]). Sliding-window KV inference occupies an interesting intermediate regime: its explicit state remains bounded, yet that state may preserve information originating far in the past through successive propagation. If such propagation is substantial, a pretrained transformer may exhibit useful recurrent memory even when only a fixed-width KV cache is retained.

2 Related Work

Information Flow and Mechanistic Interpretability Mechanistic interpretability work has explicitly attempted to reverse engineer and deconstruct transformers in order to trace information flow and better understand the inner workings of their components. Elhage et al. (Elhage et al. 2021 [5]) give a framework in which the residual stream acts as a shared communication channel that components read from and write to. Causal attention uses the residual stream to select or read information from earlier or current token positions and writes a transformed contribution to the residual stream of a destination token. Because attention can write information from earlier token representations into the residual stream, the current token representation can inherently encode more than just the raw identity of the token. Meng et al. (Meng et al. 2022 [9]) give evidence that factual information can be localized to middle-layer MLPs, and that targeted modification of these MLP mappings can alter downstream factual predictions.

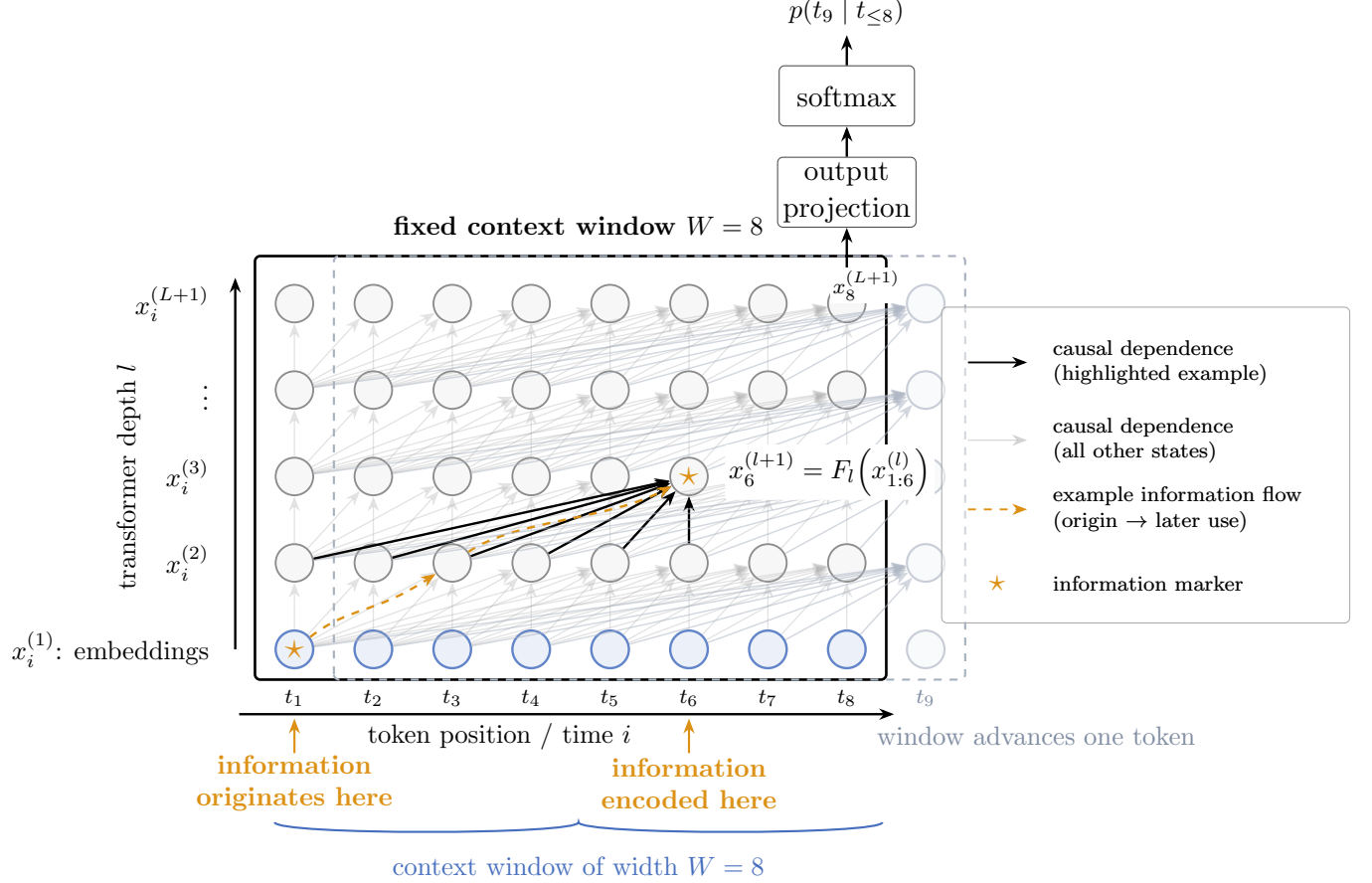


Figure 1: **Causal computation within a fixed transformer context window.** Columns index token position / time i and rows index transformer depth l . The bottom row contains token embeddings $x_i^{(1)} = \text{Embed}(t_i)$; higher rows contain successive residual-stream activations of the same dimensionality. Each activation $x_i^{(l+1)}$ is computed from $x_i^{(l)}$ together with causal attention over $x_{1:i}^{(l)}$, as defined in Section 3.1. The final activation at the most recent position, $x_8^{(L+1)}$, determines the next-token distribution through the output projection and softmax. The star marks information originating at an earlier position (here t_1) that is incorporated into a later residual-stream activation (here $x_6^{(l+1)}$) via the network’s causal computations. The faded outline indicates the same fixed-width window after advancing by one token. Notice that after the context window shifts, the origin of the information (first star) is no longer inside the new context window.

KV Caches and Long-context Inference KV caches preserve earlier computed key/value states in order to avoid later recomputation; long-context methods then optimize which cached states to keep, omit, or compress when memory is bounded. StreamingLLM (Xiao et al. 2023 [15]) is one example showing that naive sliding eviction of tokens can destabilize inference, and that retaining specific initial KV states helps. Results from the softmax attention distribution show that the initial tokens receive disproportionately high attention weights, a phenomenon the authors term *attention sinks*. StreamingLLM exploits this behavior by retaining the KV states of several initial sink tokens alongside the most recent tokens in the sliding window, which stabilizes attention computation and prevents the large increase in perplexity caused by evicting those initial states. Like StreamingLLM, H₂O (Zhang et al. 2023 [16]) takes advantage of the fact that some cached tokens matter more than others. The key observation is that a small subset of tokens receive a disproportionate amount of cumulative attention. The authors call these high-attention tokens *heavy hitters*.

Recurrent and Compressed Transformer Memory Recurrent memory approaches extend the context that is usable by carrying internal states from previous caches forward rather than recomputing them from scratch. Transformer-XL (Dai et al. 2019 [3]) caches previous hidden states and allows them to be reused in subsequent segments, enabling dependencies beyond a fixed-length context. This differs from a rolling KV setup where historical information may remain embedded in retained states after older source tokens disappear. Rae et al. (Rae et al. 2019 [12]) extend recurrent memory by compressing older hidden states into a lower-resolution memory rather than discarding them, allowing useful historical information to remain accessible over longer ranges.

More recently, state-space models such as Mamba (Gu and Dao 2023 [7]) have renewed interest in recurrent sequence models whose inference state does not grow with sequence length. At the same time, several recent production-oriented architectures have adopted hybrid designs rather than replacing attention entirely. NVIDIA’s Nemotron-H (NVIDIA 2025 [11]) replaces the majority of self-attention layers with Mamba layers while retaining periodic full-attention layers, combining constant-memory recurrent computation with direct attention over the context. Muse Glimmer (Meta AI 2026 [10]) follows a related design principle within an attention-based architecture, repeating three 2048-token sliding-window attention layers followed by one full-attention layer. These hybrid architectures illustrate practical interest in obtaining as much of the memory and inference benefit of bounded-state computation as possible while retaining periodic global access to the context.

3 Background and Inference Setup

3.1 Transformer Computation

Let $x_i^{(l)} \in \mathbb{R}^{d_{\text{model}}}$ denote the residual-stream activation at token position i at the start of layer l , with $x_i^{(1)} = \text{Embed}(t_i)$. Each layer applies causal attention over the current and preceding positions, followed by an MLP:

$$\tilde{x}_i^{(l)} = x_i^{(l)} + \text{Attn}^{(l)} \left(x_{1:i}^{(l)} \right), \quad (1)$$

$$x_i^{(l+1)} = \tilde{x}_i^{(l)} + \text{MLP}^{(l)} \left(\tilde{x}_i^{(l)} \right). \quad (2)$$

The final state at position i is mapped to vocabulary logits and a next-token distribution:

$$p(t_{i+1} \mid t_{\leq i}) = \text{softmax} \left(\text{Unembed} \left(x_i^{(L+1)} \right) \right). \quad (3)$$

Figure 1 visualizes the same computation as a grid over token position and transformer depth, with the embedding states $x_i^{(1)}$ at the bottom and causal information flow proceeding upward and to the right through successive residual-stream activations.

Notation adapted from (Arditi et al. 2024 [2]).

3.2 Information Flow in Causal Attention

The causal attention computation provides a simple way to reason about how information can be transmitted forward in token position. At every layer, there is a possible information transmission from past residual-stream activations to a later residual-stream activation. Figure 2 isolates this single-layer dependency from the larger computation shown in Figure 1.

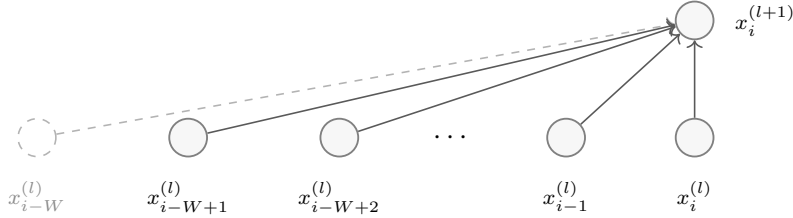


Figure 2: Single-layer information flow under causal attention with window width W . The state $x_i^{(l+1)}$ may depend on the W states $x_{i-W+1:i}^{(l)}$. The dashed state lies outside the attention window and therefore has no direct path to the target at this layer.

For a fixed attention window of width W , the state at position i and depth $l + 1$ can directly attend to the W positions

$$i - W + 1, \dots, i \quad (4)$$

of depth l .

Going from one layer to the next, information can be transmitted across at most $W - 1$ token-position steps. Using the notation of Section 3.1, the local-attention computation may be written schematically as

$$x_i^{(l+1)} = F_l \left(x_{i-W+1:i}^{(l)} \right), \quad (5)$$

where F_l represents the complete computation of layer l , including the attention operation, residual connections, and position-wise MLP.

The important point is that the inputs to F_l are not raw tokens except at the first layer. They are themselves residual-stream activations produced by the preceding layers. Each of these states may therefore already contain information originating at still earlier token positions. Information can only be transmitted across token positions through attention, but it can be transmitted again from layer l to layer $l + 1$. This layer-by-layer growth of the receptive field is also described for Mistral 7B’s sliding-window attention architecture (Jiang et al. 2023 [8]).

This gives a simple recursive description of the maximum possible information flow. Let R_l denote the greatest distance into the past from which $x_i^{(l+1)}$ can have a causal dependence after l attention layers. Before any attention layer is applied, the embedding at position i depends only on token t_i , so

$$R_0 = 0. \quad (6)$$

Each additional attention layer can extend the maximum possible receptive field by $W - 1$ token positions. Hence, for the dependency graph,

$$R_{l+1} = R_l + (W - 1), \quad (7)$$

$$\boxed{R_L = L(W - 1)}. \quad (8)$$

Equivalently, the final residual-stream activation at position i can in principle depend on input tokens as far back as

$$t_{i-L(W-1)}. \quad (9)$$

The distinction between W and $W - 1$ here is only a matter of counting positions versus distance: an attention window containing W token positions spans $W - 1$ steps from its oldest position to its newest. Thus the receptive field grows approximately as LW , but its exact maximum backward distance under this convention is $L(W - 1)$.

This is a statement about *possible causal dependence*, not a guarantee that useful information is actually transmitted. The architecture provides a path by which information can move forward through successive residual-stream activations; whether a pretrained transformer uses that path to preserve behaviorally useful information, and how much information survives as the source becomes more distant, are empirical questions.

This observation is especially relevant to sliding-window KV inference. When the window advances, previously computed states are retained rather than recomputed. A retained state may therefore carry information that originated at an earlier position which is no longer directly available in the current window. As new states are subsequently computed, that information may in principle be transmitted forward again. The experiments in Section 4 test whether this potential information flow is measurable in practice.

3.3 Rolling-KV Inference

We define Rolling-KV inference as a fixed width KV-Cache procedure where tokens are ingested sequentially such that the cache retains at most W token positions at any given time. Once the cache reaches its fixed capacity, it continues to ingest new tokens sequentially while evicting the oldest cache position. For example, if $W = 4$, after the cache contains four positions and a fifth token arrives, the first token is evicted while the 2nd through 5th remain. But the key here is that the first token is only evicted once the fifth token has been processed using the entire cache. Importantly, when the window shifts the K/V cache is retained rather than recomputed from the tokens. This is because the retained K/V states were originally computed while earlier context was still available. Therefore, the surviving cache may contain residual information influenced by earlier processed tokens that no longer explicitly remain in the cache.

4 Experiments

4.1 Experimental Setup

All three experiments evaluate the same five models: Qwen2.5-0.5B-Instruct, Qwen2.5-3B-Instruct, Mistral-7B-Instruct-v0.1, Meta-Llama-3.1-8B-Instruct, and Muse-Glimmer-30B. The first four run in BF16; Glimmer uses NF4 quantization with BF16 computation. All experiments were run on a single NVIDIA RTX 3090 GPU with 24 GB of memory. We use raw completion prompts without

Algorithm 1 Rolling-KV Inference

Require: model M , token sequence $t_{1:N}$, window width W

- 1: Process the first W tokens with model M and store the resulting K/V states as the initial cache.
- 2: **for** each remaining token position $W + 1$ through N **do**
- 3: Process the current token using the retained K/V cache.
- 4: Add the current token’s new K/V state to the cache.
- 5: If the cache now exceeds W positions, then evict the oldest cache position.
- 6: **end for**

Ensure: final K/V cache containing at most W token positions.

chat templates, neutral filler, and a retained KV window of $W = 512$ token positions. No model is trained for these experiments.

We compare three inference conditions.

1. **Rolling-KV** processes the full prompt in order, retaining at most the newest W KV positions after each step. Previously computed KV states are retained rather than recomputed when the window advances.
2. **Last Window** processes only the final W raw tokens, recomputing their states from scratch.
3. **Full Prompt** processes the entire prompt, subject to each model’s native attention rules.

For Glimmer, Rolling-KV crops the caches of both its local and global attention layers to W positions.

Each trial has four model-specific, single-token candidate answers. We score their next-token logits directly and count the trial as correct when the intended answer receives the highest score; chance accuracy is 25%. Candidates are not printed in the prompt. Experiments 1 and 2 use 100 trials per offset, model, and condition, with seed 1. Experiment 3 uses the trial counts described below.

Code, experiment configurations, raw results, and figures are available at <https://github.com/jefferythewind/Thinking-Outside-the-Box>.

4.2 Experiment 1: Secret-Code Retrieval

Task and Motivation The first experiment tests information persistence near the window boundary. Each trial introduces a secret code with the phrase “The secret code is:”, followed by filler that moves the code toward and then beyond the final raw-window boundary. At the end, the model is asked for the code, which it selects from four candidate answer tokens.

The offset is the start index of the final W -token raw window minus the code’s token index. At offset 0, the code is the first token of that raw window. At +1, it has left the final raw window but is still directly accessible during the last Rolling-KV scoring step. At +2 and beyond, its own KV state is no longer directly accessible at scoring time.

Results We sweep offsets from -5 to $+5$. At offset +1, Rolling-KV achieves 100% accuracy for each of the five models, while the equal-weight mean of the Last Window accuracies is 24.4%. This contrast occurs while the secret code’s KV state remains directly accessible to the last Rolling-KV scoring step, although the code is absent from the Last Window input.

Illustrative prompt structure

The secret code is: ER The document contains ordinary background notes about trees, roads, weather, tools, books, chairs, ... Question: What is the secret code? Answer: The boxed code is the anchor for the offset. The filler length varies to place it at different positions relative to the final raw window. Candidate answers are scored from next-token logits and are not printed in the prompt.

Table 1: Structure of a secret-code trial. The code and filler vary between trials.

At offset +2, after direct access to the code’s KV state has ended, the models diverge. Rolling-KV accuracy is 32% for Qwen 0.5B, 47% for Qwen 3B, 82% for Llama 8B, 97% for Mistral 7B, and 100% for Glimmer 30B. Mistral and Glimmer remain at or above 95% throughout the measured offsets +2 through +5, whereas the Qwen models fall substantially and Llama’s advantage diminishes with distance. Full Prompt remains near perfect across the sweep.

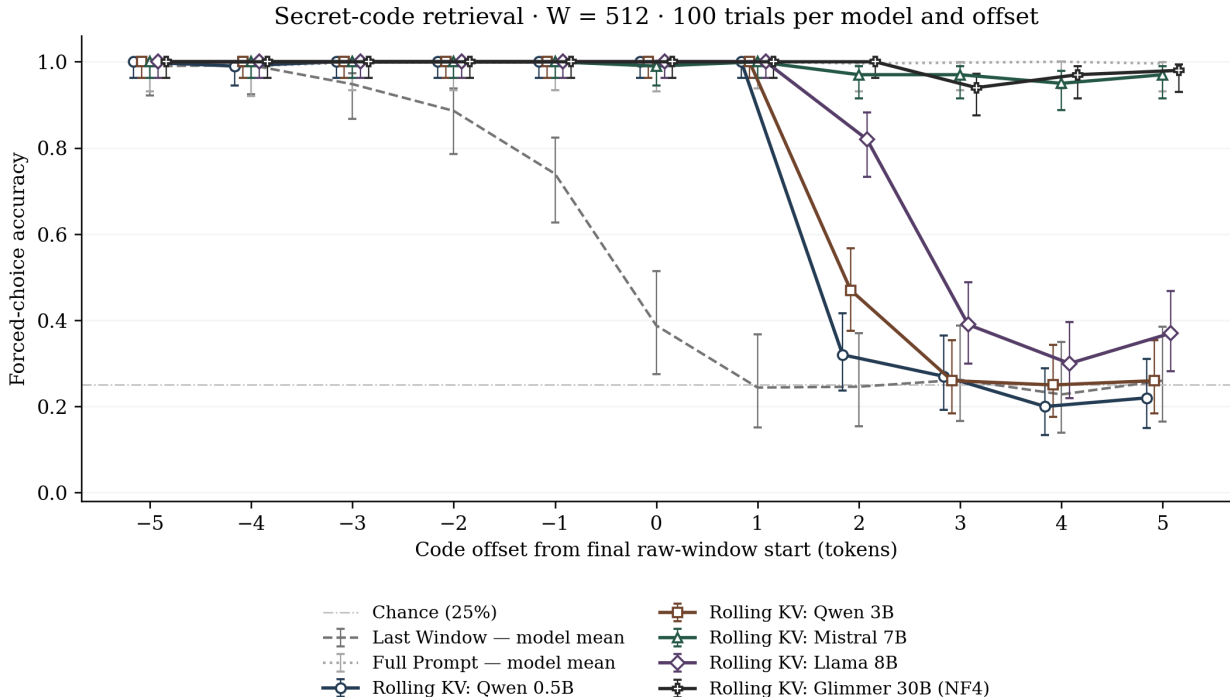


Figure 3: Secret-code retrieval accuracy across offsets -5 through $+5$, with $W = 512$. Each condition has 100 trials per offset and model. The combined control curves are equal-weight means across the five models; Rolling-KV curves are shown separately. At $+1$, the code is outside the final raw window but remains directly accessible during the last Rolling-KV scoring step. From $+2$ onward, its own KV state is no longer directly accessible. Four-choice chance accuracy is 25%.

4.3 Experiment 2: Extended Secret-Code Retrieval

Task and Motivation Experiment 2 uses the same boundary intervention but places the code after a longer defining phrase: “The secret code is defined by the following identifier:”. This moves portions of the definition out of the final raw window before the code reaches its boundary. We again ask the model to select the code from four candidate answer tokens.

Task variant	Prompt prefix
Short code	The secret code is: ER The document contains ordinary background notes ...
Long definition	The secret code is defined by the following identifier: NG The document contains ordinary background notes ...

Table 2: Secret-code definitions used in Experiments 1 and 2. Experiment 2 retains the same code-relative offset convention while lengthening the defining phrase.

Results We sweep offsets from -10 to $+10$. As the defining phrase leaves the final raw window, Last Window accuracy declines. At offset 0, its equal-weight mean across models is 40.2%, compared with 95.8% for Rolling-KV. At $+1$, the corresponding accuracies are 27.0% and 92.0%. Full Prompt remains near perfect across the sweep.

Once the code’s own KV state is no longer directly accessible, retrieval again differs by model. At $+2$, Rolling-KV achieves 30% for Qwen 0.5B, 35% for Qwen 3B, 79% for Llama 8B, 95% for Mistral 7B, and 98% for Glimmer 30B. At $+10$, Mistral and Glimmer still achieve 89% and 96%, respectively. Llama achieves 56% at that offset, while both Qwen models are near chance. Thus, the longer definition strengthens the distinction between the control and Rolling-KV before eviction, while retrieval after the code’s eviction varies substantially across models.

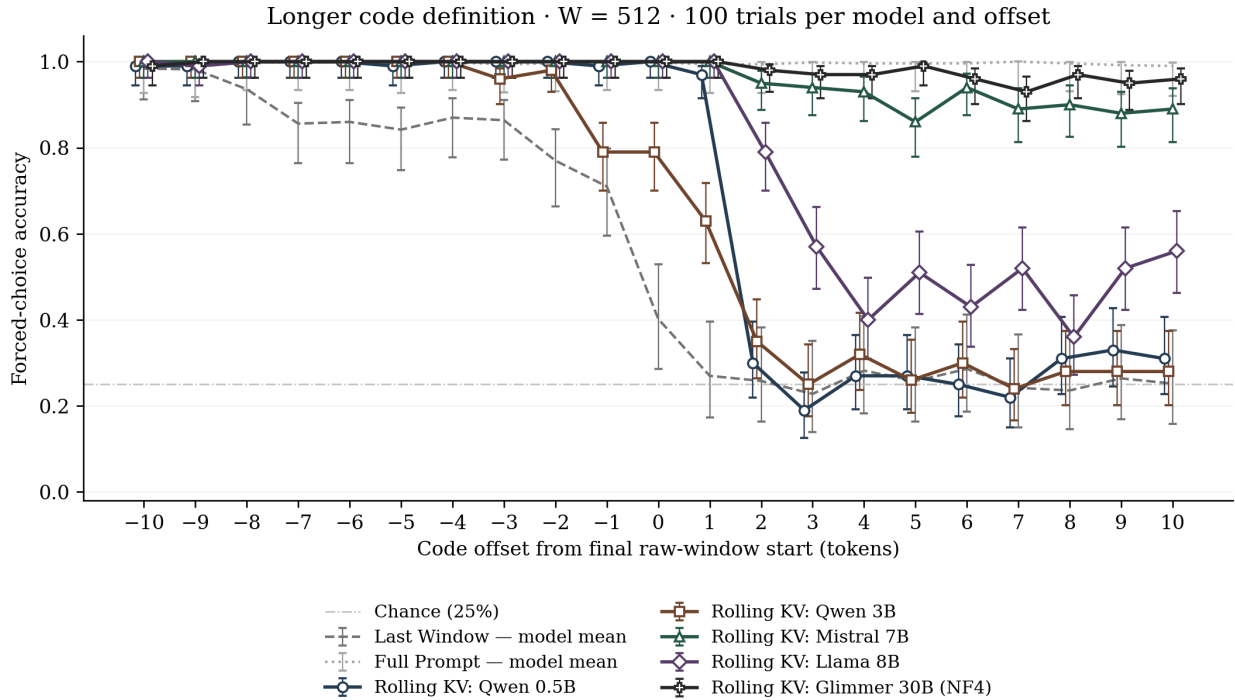


Figure 4: Forced-choice accuracy for the extended secret-code task across offsets -10 through $+10$, with $W = 512$. Each condition has 100 trials per offset and model. Combined control curves are equal-weight means across models; Rolling-KV curves are shown separately. At $+1$, the code remains directly accessible to the final Rolling-KV scoring step. At $+2$ and beyond, its own KV state is no longer directly accessible. Four-choice chance accuracy is 25%.

4.4 Experiment 3: Definition-to-Carrier Distance

Task and Motivation The first two experiments use instructions of varying length to define a secret code, then test retrieval as the definition and code move across the final context-window boundary. Experiment 3 separates the definition from the code’s later occurrence. An instruction identifies an arbitrary marker indicating where the secret will appear. After a controlled gap, the marker appears again immediately before the code. We call this later marker–code occurrence the *carrier*.

This construction tests two distances independently: the gap from the definition to the carrier, and the distance from the code to the final window boundary. In particular, it lets us ask whether the earlier definition remains useful when it is outside the window available as the carrier is processed, and whether the answer remains recoverable after the code itself has also been evicted.

Illustrative prompt structure

This is a memory test.

The secret code will appear later in this text.

The secret is the value immediately following <Q3>.

... [definition-to-carrier filler] ...

<Q3>: NULL

... [tail filler] ...

Question: What is the secret code?

Answer:

The definition-to-carrier gap counts filler tokens after the defining preamble and before the later marker prefix. Tail filler sets the boxed code’s position relative to the final raw window. Candidate answers are scored from next-token logits; they are not listed in the prompt.

Table 3: Structure of an Experiment 3 trial, illustrated with a marker and answer found in the saved context examples. The marker and code vary between trials.

Experimental Construction Each prompt begins with 128 filler tokens followed by the defining preamble. We vary the number of filler tokens between the end of that preamble and the carrier prefix over $D \in \{32, 64, 128, 256, 512, 768, 1024\}$. The tokenized marker prefix adds a few more tokens between the preamble and the code.

Independently, we vary the code-relative boundary offset over $\{-32, 0, 32, 64, 128, 256\}$. The offset is the start index of the final W -token raw window minus the code’s index. Thus, at -32 the code is 32 tokens inside that window; at 0 it is the window’s first token. At the positive offsets in this sweep, the code is absent from both the final raw window and the retained Rolling-KV positions at scoring time.

For each model and grid cell, Rolling-KV is evaluated on 100 trials. The Last Window and Full Prompt controls each use 20 prompts sampled from those saved Rolling-KV trials per model and cell. Figure 5 displays the two controls as equal-weight means across models and the Rolling-KV results separately for each model.

Results Full Prompt accuracy remains high across the grid, indicating that the task is generally solvable when the definition and code are available together. Once the code is absent from the final raw window, Last Window accuracy is generally near the four-choice chance level of 25%.

The dotted lines in Figure 5 divide each heat map into four regions. The horizontal line falls between offsets 0 and 32: below it, the code is visible in the final raw window; above it, the code

has been evicted. The vertical line falls between gaps of 256 and 512 filler tokens, separating gaps shorter than W from gaps of at least W .

In the bottom-left region, the code is visible at the final step and the definition-to-carrier filler gap is shorter than one window. Four of the five models perform strongly under Rolling-KV, although Qwen 3B is notably weaker than Qwen 0.5B. In the bottom-right region, the code is still visible but the gap is at least one window long. Mistral 7B and Glimmer 30B remain highly accurate, while the other three models decline.

The upper regions test the stronger case in which the code itself has been evicted. In the top-left region, the filler gap is shorter than one window: Glimmer averages 95.4% across these cells, while Mistral averages 54.2%. The top-right region combines an evicted code with a gap of at least 512 filler tokens. It is the hardest setting, yet Glimmer averages 62.9% and Mistral 39.9%; the other three models remain close to chance. These regional means summarize the displayed cells, rather than a separately sampled test.

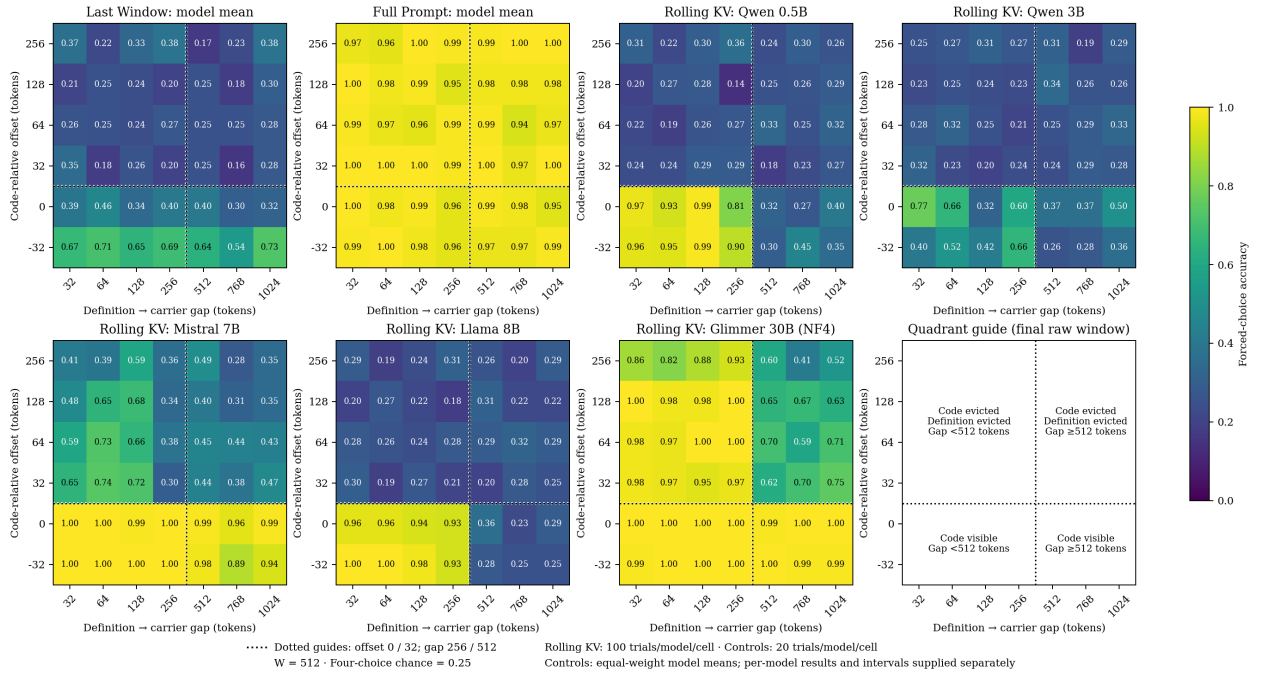


Figure 5: Experiment 3 forced-choice accuracy over the definition-to-carrier filler gap (horizontal axis) and the code-relative boundary offset (vertical axis), with $W = 512$. The top two panels show equal-weight means of the five model-specific controls; the remaining panels show Rolling-KV accuracy for each model. Each Rolling-KV cell contains 100 trials per model, and each control cell contains 20 trials per model. Dotted lines separate offsets 0 and 32, and gaps 256 and 512. Positive offsets shown here place the code outside the final raw window and retained cache. Four-choice chance accuracy is 25%.

5 Discussion

The experiments address the central question of this study: whether information remains usable after its source leaves a rolling KV window, and for how long. They do not characterize all information transmitted through the cache, but they reveal three settings in which Rolling-KV

retrieves a secret code that the Last Window control cannot reliably identify. Experiments 1 and 2 probe relatively short distances around the window boundary. Experiment 3 tests whether useful information can persist across a definition-to-carrier gap extending beyond the retained window size, as the dependency analysis in Section 3.2 suggests is possible.

Contextualized Representations as Carriers The first mode occurs when the secret code remains visible near the end of the prompt but its defining instruction, such as “The secret code is:”, does not. Rolling-KV processed the code while that instruction was available, so the retained KV states associated with the code were computed in its context. Last Window sees the code without that earlier definition and cannot reliably identify it as the requested answer. We call this mode *contextualized representations as carriers*. All five models benefit from this mode in the tested conditions.

Latent Information Relay The second mode occurs when both the defining instruction and the code itself have left the final window. The Last Window input then contains neither the answer nor the information identifying it as the secret. Some models nevertheless recover the code with Rolling-KV. In Experiments 1 and 2, Llama 3.1 shows substantial retrieval immediately after the code’s KV state is no longer directly accessible, but its accuracy declines at larger offsets. Mistral 7B and Muse Glimmer sustain high accuracy across the measured post-eviction offsets, whereas the Qwen2.5 models decline much more sharply.

The successful Rolling-KV retrieval shows that information from the earlier prompt remains usable through retained states after the code’s own KV entries have been evicted. We call this mode *latent information relay*. It is stronger than retrieval from a still-cached code representation because a later retained state must support the answer. This suggests a route toward recurrent-style memory during streaming inference, although these experiments do not identify the specific states or computations carrying the information.

Long-Range Transmission Experiment 3 extends this question by placing filler between the definition and the later marker-code carrier. When this gap reaches or exceeds the 512-token window, the defining instruction is no longer directly available as the carrier is processed. Mistral and Glimmer remain highly accurate in this setting while the code is still retained. Glimmer also retains above-chance retrieval in the hardest region, where the gap is at least 512 tokens and the code itself has subsequently been evicted. These results show that useful information can cross a window-scale gap and remain available beyond the code’s own eviction. Performance varies substantially by model and by distance, so the existence of this path does not imply reliable long-term memory in general.

Limitations Our evidence is behavioral: a model produces the correct answer despite the source tokens being unavailable for direct attention at the final step. It does not show how much of the original information survives or whether the evicted text could be reconstructed. We therefore distinguish task-relevant preservation from lossless memory. The retained states need only carry consequences of the earlier context sufficient for this retrieval task.

Failure to retrieve the code also does not prove that no information about it remains. A retained state might contain information that the model cannot use to answer the particular question. Conversely, successful retrieval provides evidence that some surviving information is functionally accessible, without identifying its location or representation in the cache. Direct interventions on retained states would be needed to examine that mechanism more closely.

The experiments use a synthetic four-choice task, a fixed $W = 512$ cache, and a small set of models. Accuracy on this task does not establish how much information a rolling cache can preserve in natural dialogue, nor how long it remains useful as more content is ingested. The models differ in size, architecture, training, and precision, so their relative performance cannot isolate the cause of their different behavior. Finally, the experiments measure retrieval rather than directly tracing information through individual layers or cached states.

Future Work These experiments show that usable information can survive beyond a fixed rolling KV window, but they do not show that a model will reliably use this path. Mistral 7B and Muse Glimmer, the two models with the strongest retrieval after code eviction, include local sliding attention in their published architectures. The Qwen2.5 models tested here disable sliding attention, while Llama 3.1 uses a standard attention architecture. This association is suggestive, but the models differ in many other respects, and their published descriptions do not establish that any was trained specifically to preserve information after KV eviction.

A more direct test would train matched models with the same architecture and data, varying whether training includes a rolling cache and tasks whose relevant information must cross one or more window boundaries. Evaluation on new markers, content, and distances could then test whether such training improves generalizable long-term retrieval. Interventions on the retained KV states could also help identify where task-relevant information travels and when it becomes inaccessible.

Safety A safety motivation for this work is understanding what information a model can still use after its source has left the visible context. Directly questioning a model can provide behavioral evidence about information available from its internal processing. Fornasiere et al. found that language models could identify where their activations had been perturbed and, in some settings, distinguish the type of perturbation (Fornasiere et al. 2026 [6]). Our experiments similarly ask models to retrieve information encountered earlier during inference. Successful retrieval shows that the information remains usable for this task, though it does not reveal where or how it is represented.

Identifying the path that carries this information requires a different kind of investigation. Anthropic’s circuit-tracing work maps features and their interactions during particular model computations (Ameisen et al. 2025 [1]). Applying such methods to rolling inference could help trace how information from an evicted source token influences later retained states and, ultimately, the answer. This would connect the behavioral effects observed here to a more specific internal mechanism.

Author Contributions

Tim led the research, designed and implemented the experiments, analyzed the results, and drafted the manuscript. Seth contributed to the development and discussion of the ideas, wrote substantial portions of the several sections, ran experiments, and revised the manuscript. Both authors reviewed and approved the final version.

AI-Assisted Tools

The research questions, experimental design, results, and scientific interpretation were developed by the authors. ChatGPT was used to assist with manuscript editing, proofreading, and LaTeX for-

matting, and Codex was used for coding assistance. The authors reviewed the resulting manuscript and code and take responsibility for their content.

Code and Data Availability

Code, experiment configurations, raw trial results, and figures are available at <https://github.com/jefferythewind/Thinking-Outside-the-Box>.

References

- Ameisen, Emmanuel, Jack Lindsey, Adam Pearce, et al. (2025). “Circuit Tracing: Revealing Computational Graphs in Language Models”. In: *Transformer Circuits Thread*. URL: <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- Arditi, Andy, Oscar Obeso, Aaquib Syed, Daniel Paleka, Nina Panickssery, Wes Gurnee, and Neel Nanda (2024). *Refusal in Language Models Is Mediated by a Single Direction*. arXiv: 2406.11717 [cs.LG]. URL: <https://arxiv.org/abs/2406.11717>.
- Dai, Zihang, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov (2019). “Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context”. In: *arXiv preprint arXiv:1901.02860*.
- Dubey, Abhimanyu, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. (2024). “The Llama 3 Herd of Models”. In: *arXiv preprint arXiv:2407.21783*. URL: <https://arxiv.org/abs/2407.21783>.
- Elhage, Nelson, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah (2021). “A Mathematical Framework for Transformer Circuits”. In: *Transformer Circuits Thread*. URL: <https://transformer-circuits.pub/2021/framework/index.html>.
- Fornasiere, Damiano, Mirko Bronzi, Spencer Kitts, Alessandro Palmas, Yoshua Bengio, and Oliver Richardson (2026). *Language Models Recognize Dropout and Gaussian Noise Applied to Their Activations*. arXiv: 2604.17465 [cs.AI]. URL: <https://arxiv.org/abs/2604.17465>.
- Gu, Albert and Tri Dao (2023). “Mamba: Linear-Time Sequence Modeling with Selective State Spaces”. In: *arXiv preprint arXiv:2312.00752*.
- Jiang, Albert Q., Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed (2023). “Mistral 7B”. In: *arXiv preprint arXiv:2310.06825*.
- Meng, Kevin, David Bau, Alex Andonian, and Yonatan Belinkov (2022). “Locating and Editing Factual Associations in GPT”. In: *arXiv preprint arXiv:2202.05262*.
- Meta AI (Aug. 2026). *Introducing Muse Glimmer: An Open Agentic Model That Runs on Your Device*. <https://research.meta.ai/blog/introducing-muse-glimmer-open-agentic-model/>. Accessed: 2026-09-09.
- NVIDIA (2025). “Nemotron-H: A Family of Accurate and Efficient Hybrid Mamba-Transformer Models”. In: *arXiv preprint arXiv:2504.03624*.
- Rae, Jack W., Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap (2019). “Compressive Transformers for Long-Range Sequence Modelling”. In: *arXiv preprint arXiv:1911.05507*.

- Team, Qwen (2025). *Qwen2.5 Technical Report*. arXiv: 2412.15115 [cs.CL]. URL: <https://arxiv.org/abs/2412.15115>.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin (2017). “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30.
- Xiao, Guangxuan, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis (2023). “Efficient Streaming Language Models with Attention Sinks”. In: *arXiv preprint arXiv:2309.17453*.
- Zhang, Zhenyu, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen (2023). “H₂O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models”. In: *arXiv preprint arXiv:2306.14048*.